HACKER WOMEN COUNCIL

SQL

& WEB SECURITY

CHALLENGE

Parte 1: SQL Easy
Documentación de los 31 ejercicios

Parte 2: OWASP ZAP
Primera evaluación de seguridad web de Gin & Juice Shop

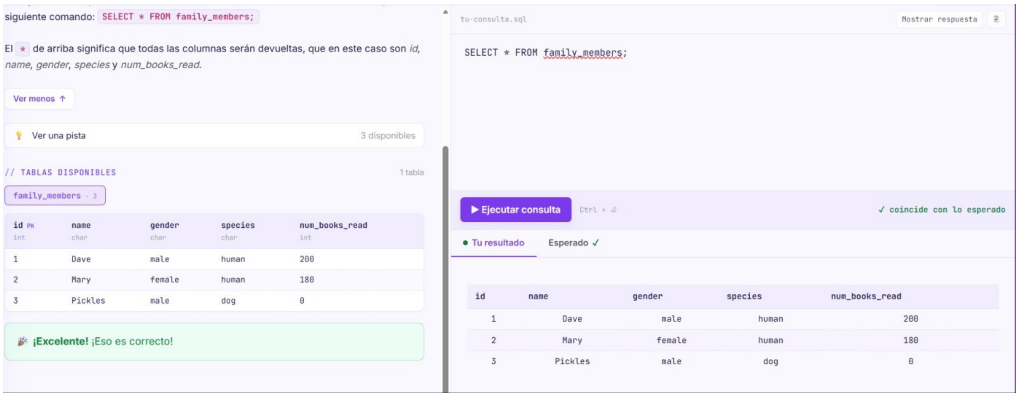
Natacha Yael Lucas

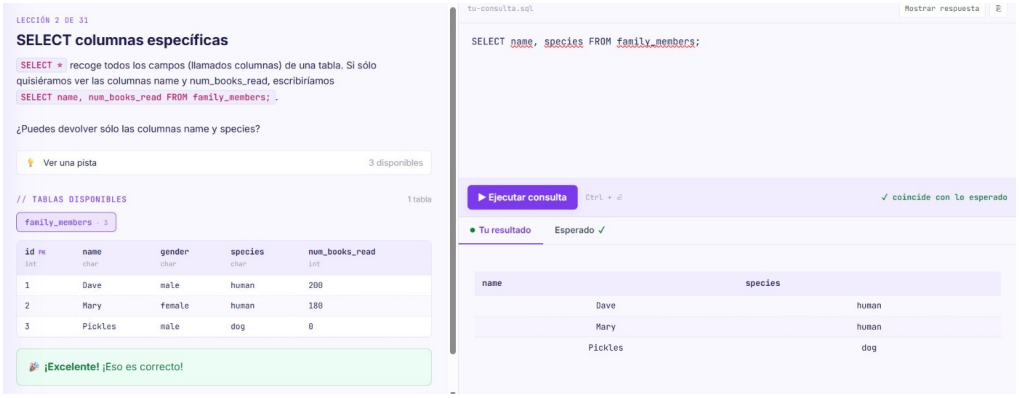
FECHA DE ENTREGA: **27/10/2026**

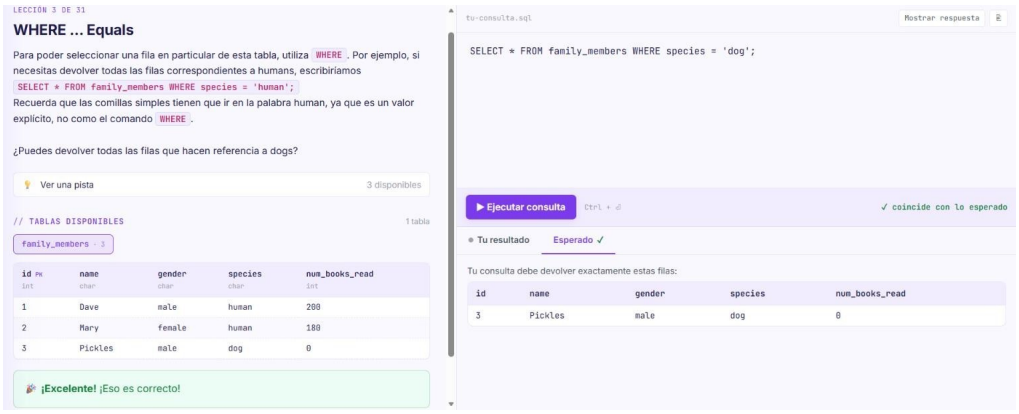
Parte 1: SQL Easy, los 31 ejercicios

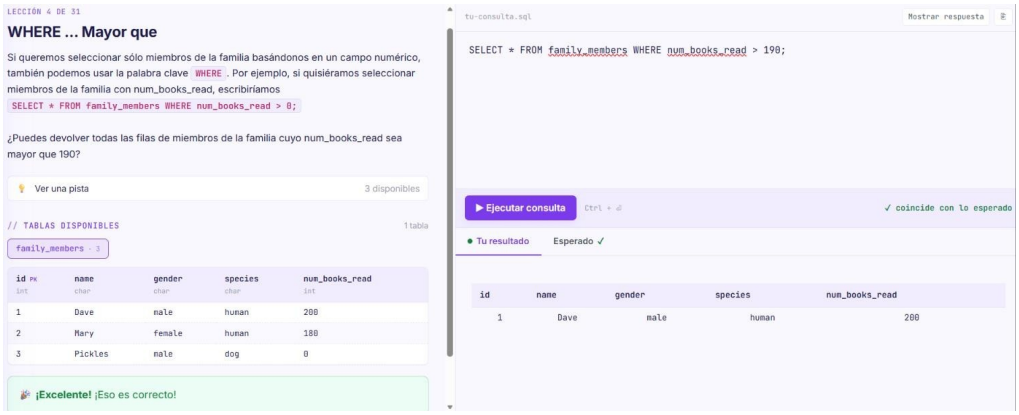
A continuación documento cada uno de los 31 ejercicios de SQL Easy con el formato de la guía: objetivo, concepto, consulta ejecutada, captura del resultado, interpretación, aprendizaje y aplicación en el mundo real. Los ejercicios siguen el orden de la plataforma y están agrupados en cuatro bloques: Fundamentos (1 a 11), Agregaciones (12 a 20), Joins (21 a 27) y Funciones de texto y lógica (28 a 31).

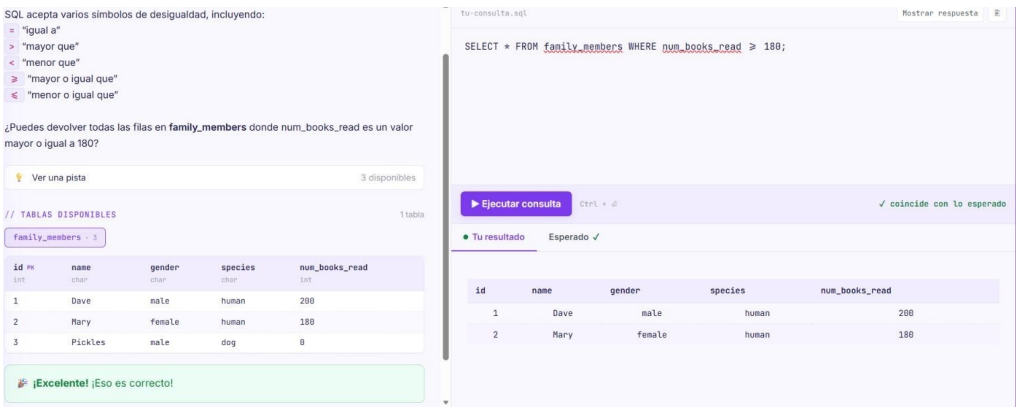
Bloque: Fundamentos

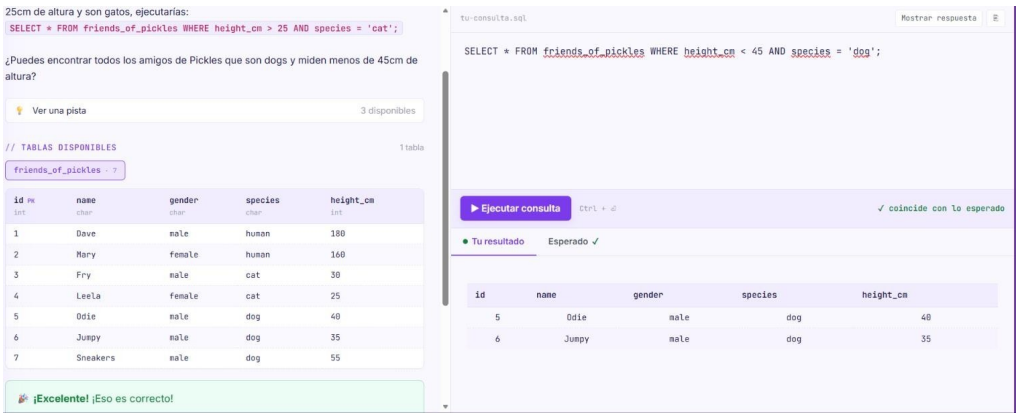
01	Ejercicio 1: SELECT *
02 Objetivo	Ver todos los datos de la tabla family_members.
03 Concepto SQL	SELECT con * para traer todas las columnas de una tabla.
04 Consulta	`SELECT * FROM family_members;`
05 Resultado	
06 Interpretación	Devuelve las 3 filas (Dave, Mary y Pickles) con sus 5 columnas: id, name, gender, species y num_books_read.
07 Aprendizaje	El asterisco es un atajo para pedir todas las columnas; sirve para conocer cómo es una tabla antes de filtrarla.
08 Aplicación en el mundo real	Un analista lo usa para echar un primer vistazo a una tabla nueva, por ejemplo la de clientes de una empresa.

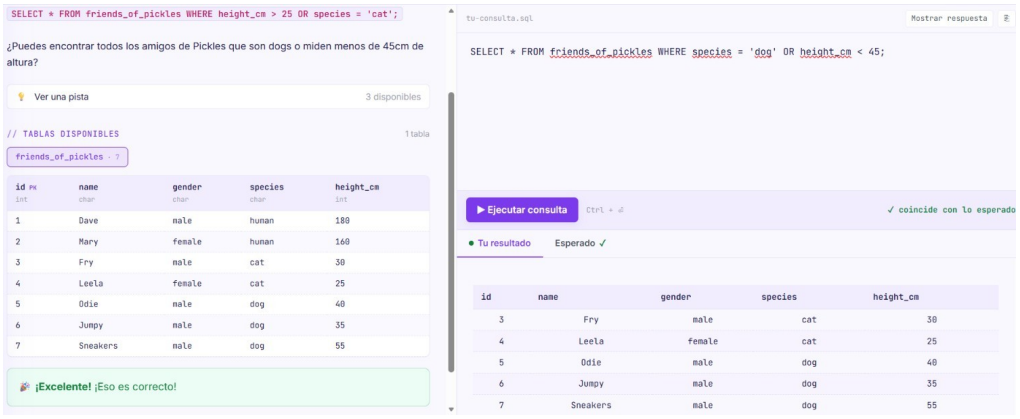
01	Ejercicio 2: SELECT columnas específicas
02 Objetivo	Mostrar solo el nombre y la especie de cada miembro.
03 Concepto SQL	SELECT con una lista de columnas separadas por coma.
04 Consulta	`SELECT name, species FROM family_members;`
05 Resultado	
06 Interpretación	La tabla resultante tiene solo dos columnas (name y species) para las 3 filas.
07 Aprendizaje	Es mejor pedir solo las columnas necesarias: el resultado es más claro y se mueven menos datos.
08 Aplicación en el mundo real	Sacar solo nombre y correo de una base de clientes para una campaña, sin exponer el resto de los datos.

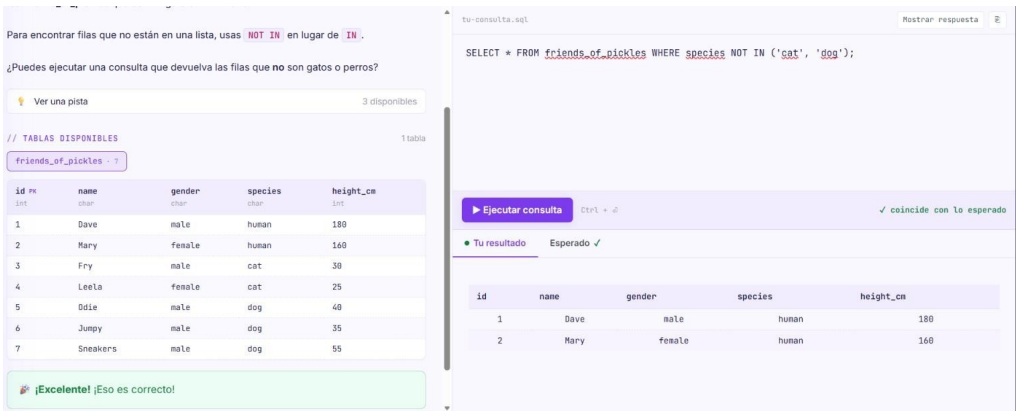
01	Ejercicio 3: WHERE ... Equals
02 Objetivo	Obtener las filas cuya especie es perro.
03 Concepto SQL	WHERE con el operador = sobre un valor de texto.
04 Consulta	`SELECT * FROM family_members WHERE species = 'dog';`
05 Resultado	
06 Interpretación	Aparece una sola fila: Pickles, que es el único perro de la tabla.
07 Aprendizaje	Los valores de texto van entre comillas simples; los nombres de columnas y comandos, sin comillas.
08 Aplicación en el mundo real	Buscar en una tienda todos los productos de una categoría, o los tickets de soporte de un cliente puntual.

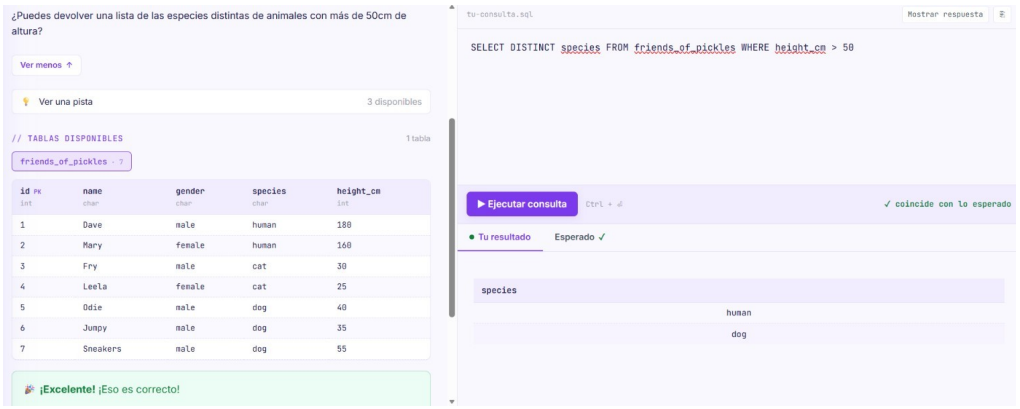
01	Ejercicio 4: WHERE ... Mayor que
02 Objetivo	Encontrar quién leyó más de 190 libros.
03 Concepto SQL	WHERE con el operador > sobre un campo numérico.
04 Consulta	`SELECT * FROM family_members WHERE num_books_read > 190;`
05 Resultado	
06 Interpretación	Solo Dave cumple la condición, con 200 libros leídos. Mary (180) queda afuera.
07 Aprendizaje	El filtro > es estricto: el valor justo en el límite no entra.
08 Aplicación en el mundo real	Listar las ventas que superan cierto monto o los usuarios con más de X intentos fallidos de ingreso.

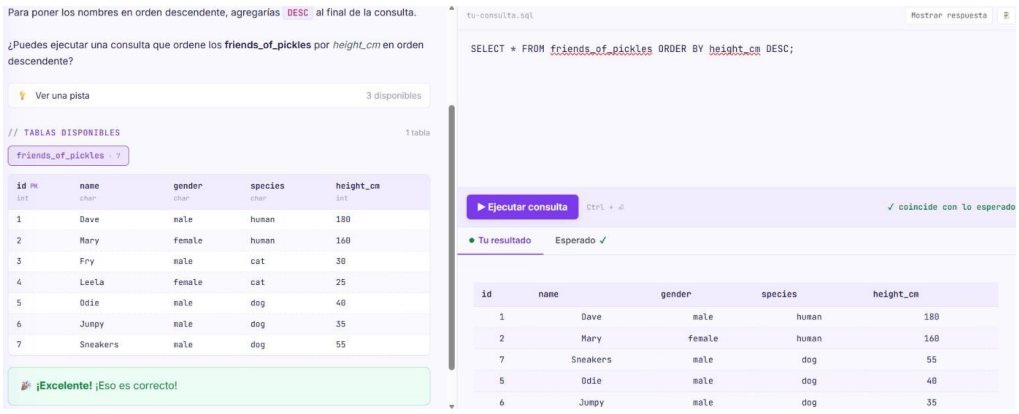
01	Ejercicio 5: WHERE ... Mayor o igual que
02 Objetivo	Obtener quienes leyeron 180 libros o más.
03 Concepto SQL	Operadores de comparación: =, >, <, >= y <=.
04 Consulta	`SELECT * FROM family_members WHERE num_books_read >= 180;`
05 Resultado	
06 Interpretación	Devuelve a Dave (200) y a Mary (180). Al usar >= ahora Mary sí entra.
07 Aprendizaje	Comparé con la lección anterior y vi que la diferencia entre > y >= está en incluir o no el valor del límite.
08 Aplicación en el mundo real	Seleccionar estudiantes que alcanzaron la nota mínima para aprobar o clientes que llegaron a un puntaje.

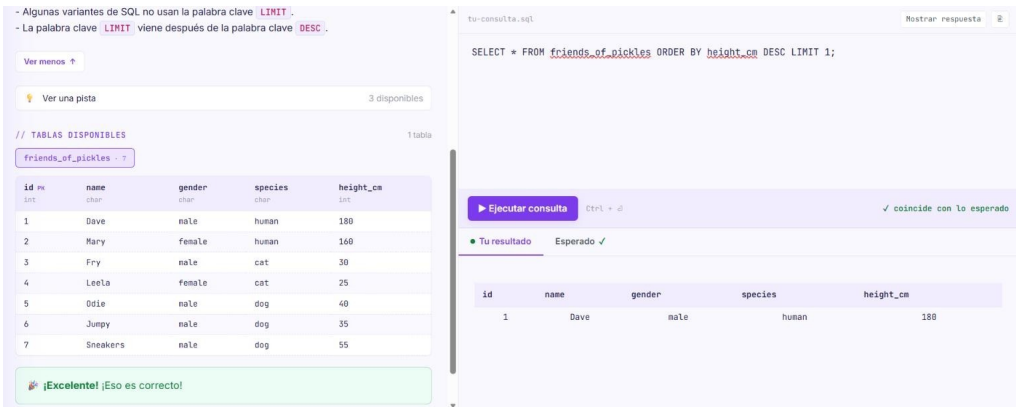
01	Ejercicio 6: AND
02 Objetivo	Hallar los amigos de Pickles que son perros y miden menos de 45 cm.
03 Concepto SQL	Combinar condiciones en WHERE con AND (deben cumplirse todas).
04 Consulta	`SELECT * FROM friends_of_pickles WHERE height_cm < 45 AND species = 'dog';`
05 Resultado	
06 Interpretación	Devuelve a Odie (40 cm) y Jumpy (35 cm). Sneakers también es perro pero mide 55, así que queda fuera.
07 Aprendizaje	Con AND cada condición adicional reduce el resultado, porque la fila tiene que cumplir las dos.
08 Aplicación en el mundo real	Filtrar usuarios que están activos Y pertenecen a cierto país, o transacciones sospechosas por monto Y horario.

01	Ejercicio 7: OR
02 Objetivo	Encontrar los amigos que son perros o miden menos de 45 cm.
03 Concepto SQL	Combinar condiciones con OR (alcanza con que se cumpla una).
04 Consulta	`SELECT * FROM friends_of_pickles WHERE species = 'dog' OR height_cm < 45;`
05 Resultado	
06 Interpretación	Devuelve 5 filas: Fry, Leela, Odie, Jumpy y Sneakers. Fry y Leela entran por altura; Sneakers, por ser perro.
07 Aprendizaje	OR amplía el resultado, al revés que AND: basta con que una de las condiciones sea verdadera.
08 Aplicación en el mundo real	Buscar clientes que compraron un producto o que se suscribieron al boletín, para armar una lista de contacto.

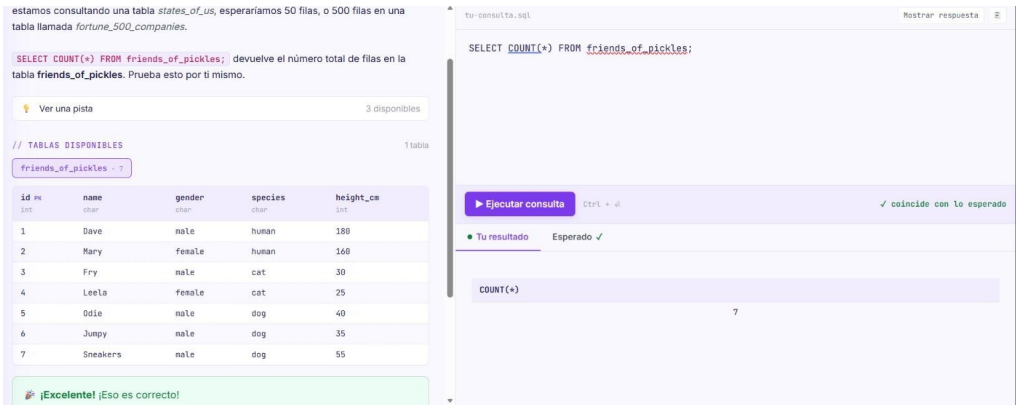
01	Ejercicio 8: IN
02 Objetivo	Obtener a los que no son gatos ni perros.
03 Concepto SQL	IN y NOT IN para comparar contra una lista de valores.
04 Consulta	`SELECT * FROM friends_of_pickles WHERE species NOT IN ('cat', 'dog');`
05 Resultado	
06 Interpretación	Devuelve a Dave y Mary, los dos humanos, porque son los únicos que no están en la lista.
07 Aprendizaje	NOT IN evita encadenar varios != con AND y deja la consulta más corta y fácil de leer.
08 Aplicación en el mundo real	Excluir una lista de países o de IP bloqueadas al revisar los accesos a un sistema.

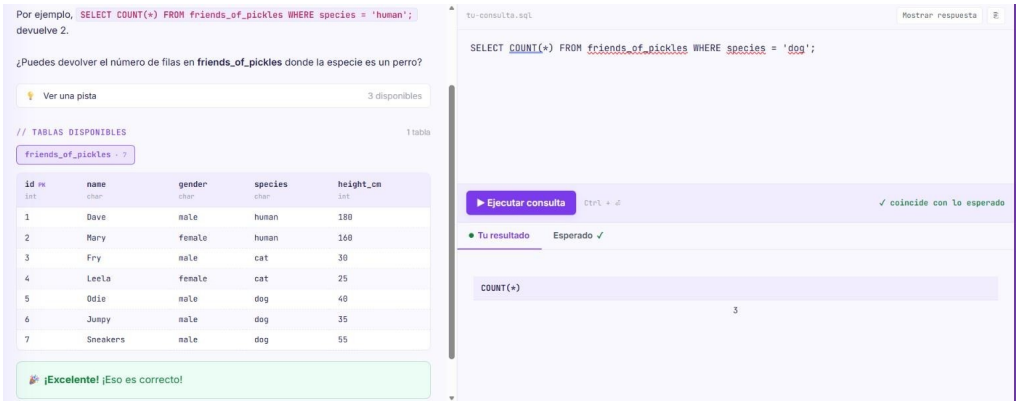
01	Ejercicio 9: DISTINCT
02 Objetivo	Listar las especies distintas de los animales de más de 50 cm.
03 Concepto SQL	DISTINCT para eliminar filas repetidas en el resultado.
04 Consulta	`SELECT DISTINCT species FROM friends_of_pickles WHERE height_cm > 50`
05 Resultado	
06 Interpretación	Devuelve solo human y dog, aunque hay varios perros y humanos que cumplen: cada especie aparece una vez.
07 Aprendizaje	DISTINCT sirve para saber qué valores distintos existen en una columna, sin ver repeticiones.
08 Aplicación en el mundo real	Obtener la lista de ciudades distintas desde donde compran los clientes, o los tipos de alerta distintos de un sistema.

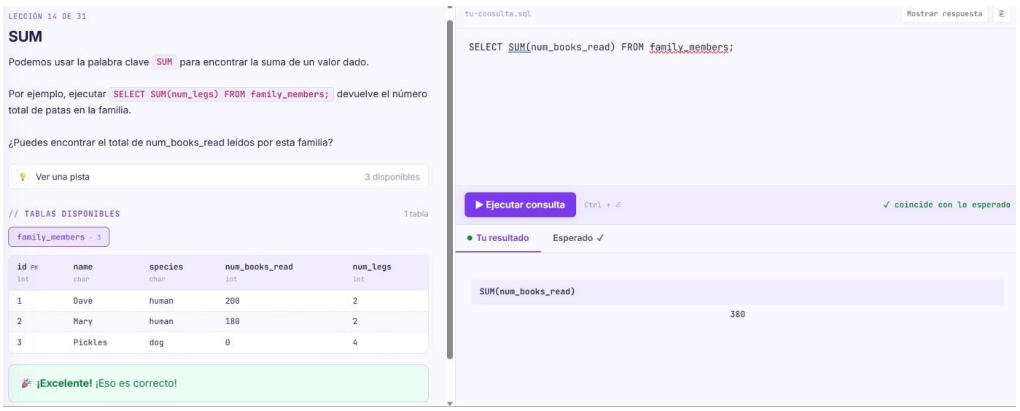
01	Ejercicio 10: ORDER BY
02 Objetivo	Ordenar a los amigos de Pickles de más alto a más bajo.
03 Concepto SQL	ORDER BY con DESC para orden descendente.
04 Consulta	`SELECT * FROM friends_of_pickles ORDER BY height_cm DESC;`
05 Resultado	
06 Interpretación	Quedan primero Dave (180), luego Mary (160), Sneakers (55), Odie (40) y así hasta Leela (25).
07 Aprendizaje	Sin DESC el orden es ascendente por defecto; con DESC se invierte.
08 Aplicación en el mundo real	Mostrar los productos más vendidos primero o ordenar los incidentes de seguridad por gravedad.

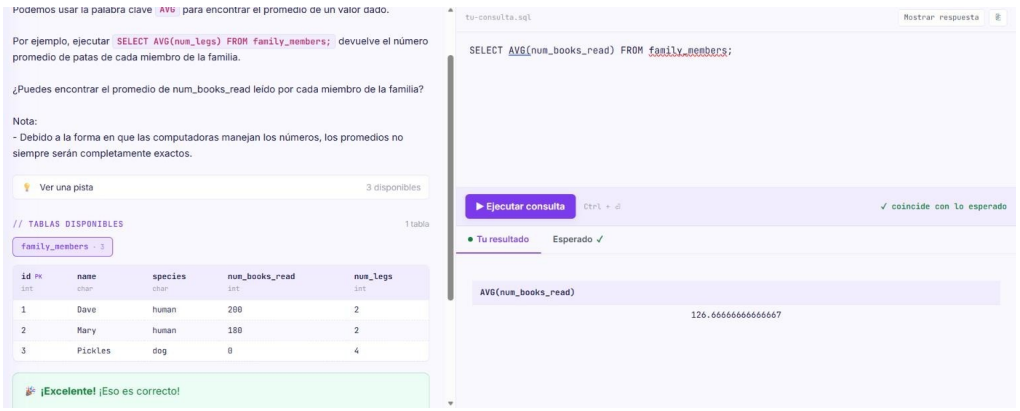
01	Ejercicio 11: LIMIT # de filas devueltas
02 Objetivo	Obtener solo el amigo de Pickles más alto, con todas sus columnas.
03 Concepto SQL	LIMIT combinado con ORDER BY ... DESC.
04 Consulta	`SELECT * FROM friends_of_pickles ORDER BY height_cm DESC LIMIT 1;`
05 Resultado	
06 Interpretación	Devuelve una sola fila: Dave, con 180 cm, que es el valor más alto.
07 Aprendizaje	LIMIT va al final de la consulta, después de ORDER BY y DESC; el orden de las cláusulas importa.
08 Aplicación en el mundo real	Armar un ranking tipo "top 10" o mirar solo las últimas filas de una tabla enorme sin cargar todo.

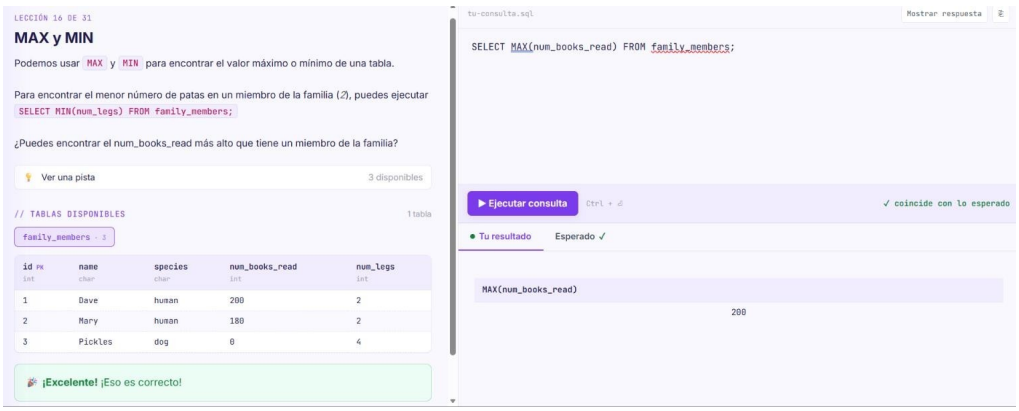
Bloque: Agregaciones

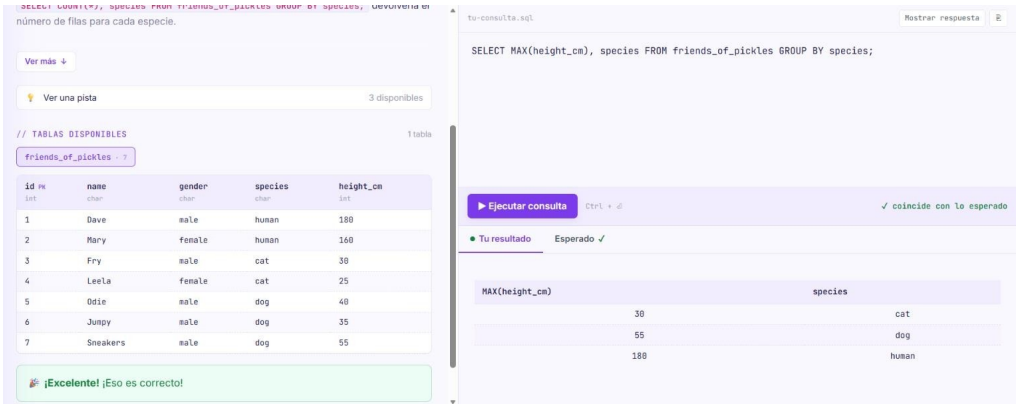
01	Ejercicio 12: COUNT(*)
02 Objetivo	Saber cuántas filas tiene la tabla friends_of_pickles.
03 Concepto SQL	Función de agregación COUNT(*).
04 Consulta	`SELECT COUNT(*) FROM friends_of_pickles;`
05 Resultado	
06 Interpretación	El resultado es una sola celda con el número 7: la tabla tiene 7 filas.
07 Aprendizaje	Las funciones de agregación devuelven un único valor que resume muchas filas.
08 Aplicación en el mundo real	Contar cuántos registros hay en una tabla de logs o cuántos usuarios tiene una aplicación.

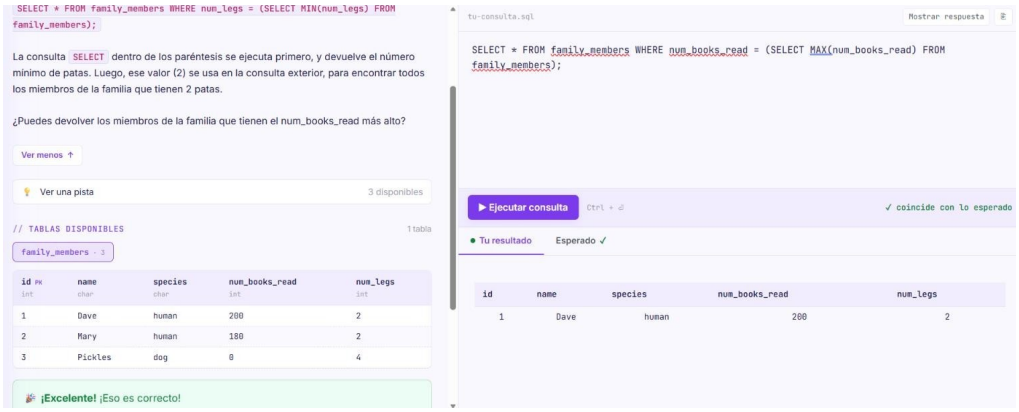
01	Ejercicio 13: COUNT(*) ... WHERE
02 Objetivo	Contar cuántos de los amigos son perros.
03 Concepto SQL	COUNT(*) combinado con WHERE.
04 Consulta	`SELECT COUNT(*) FROM friends_of_pickles WHERE species = 'dog';`
05 Resultado	
06 Interpretación	Devuelve 3, que coincide con Odie, Jumpy y Sneakers.
07 Aprendizaje	Primero se filtran las filas con WHERE y después se cuentan las que quedaron.
08 Aplicación en el mundo real	Contar los intentos fallidos de inicio de sesión de un usuario en el último día.

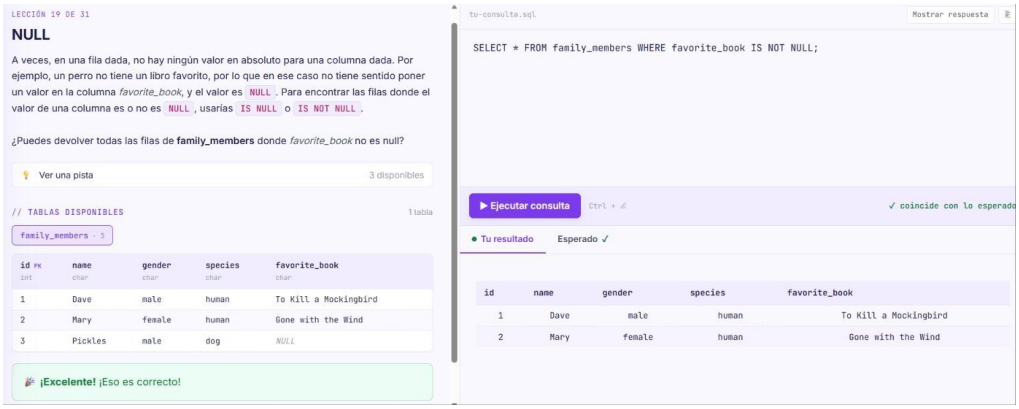
01	Ejercicio 14: SUM
02 Objetivo	Calcular el total de libros leídos por la familia.
03 Concepto SQL	Función de agregación SUM.
04 Consulta	`SELECT SUM(num_books_read) FROM family_members;`
05 Resultado	
06 Interpretación	El resultado es 380, que sale de sumar 200 + 180 + 0.
07 Aprendizaje	SUM solo tiene sentido con columnas numéricas y suma todas las filas, incluidas las que valen 0.
08 Aplicación en el mundo real	Calcular la facturación total de un mes sumando todas las ventas.

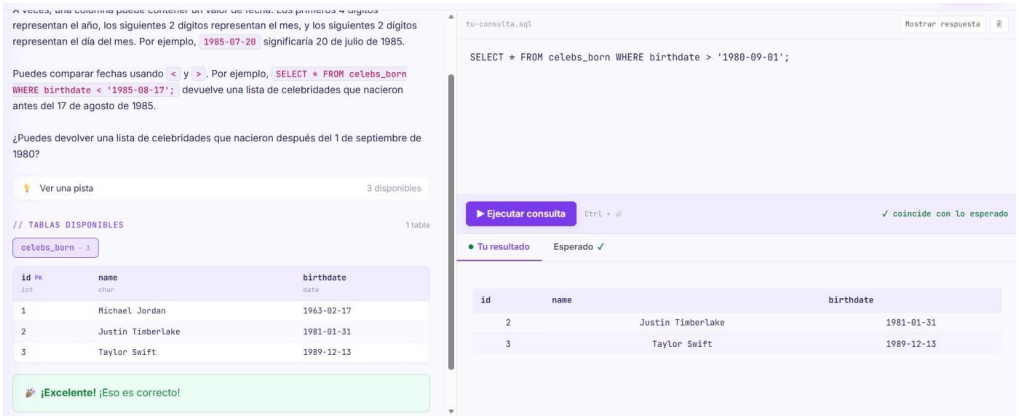
01	Ejercicio 15: AVG
02 Objetivo	Calcular el promedio de libros leídos por cada miembro.
03 Concepto SQL	Función de agregación AVG.
04 Consulta	`SELECT AVG(num_books_read) FROM family_members;`
05 Resultado	
06 Interpretación	Devuelve 126,67 aproximadamente (380 dividido 3 miembros), con varios decimales.
07 Aprendizaje	El promedio incluye a Pickles, que leyó 0, y por eso baja el resultado; AVG cuenta todas las filas.
08 Aplicación en el mundo real	Obtener el tiempo promedio de respuesta de un servidor o la nota promedio de un curso.

01	Ejercicio 16: MAX y MIN
02 Objetivo	Encontrar la mayor cantidad de libros leídos por un miembro.
03 Concepto SQL	Funciones MAX y MIN.
04 Consulta	`SELECT MAX(num_books_read) FROM family_members;`
05 Resultado	
06 Interpretación	El resultado es 200, el valor más alto de la columna.
07 Aprendizaje	MAX devuelve el valor, no la fila completa; para saber quién es hay que combinarlo con otra consulta.
08 Aplicación en el mundo real	Conocer el pico máximo de tráfico en un sitio web o el precio más bajo de un producto.

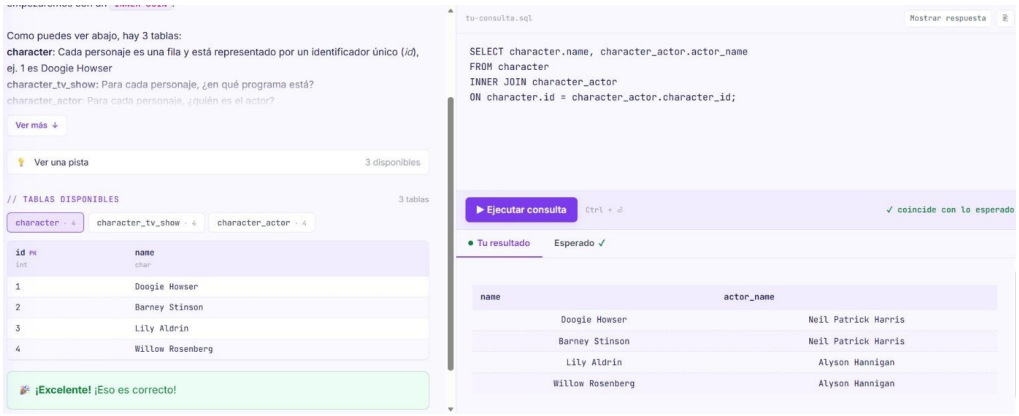
01	Ejercicio 17: GROUP BY
02 Objetivo	Obtener la altura máxima de cada especie.
03 Concepto SQL	GROUP BY junto con una función de agregación.
04 Consulta	`SELECT MAX(height_cm), species FROM friends_of_pickles GROUP BY species;`
05 Resultado	
06 Interpretación	Devuelve una fila por especie: cat 30, dog 55 y human 180.
07 Aprendizaje	GROUP BY divide la tabla en grupos y la función de agregación se calcula dentro de cada uno.
08 Aplicación en el mundo real	Ver las ventas totales por sucursal o la cantidad de alertas por tipo de amenaza.

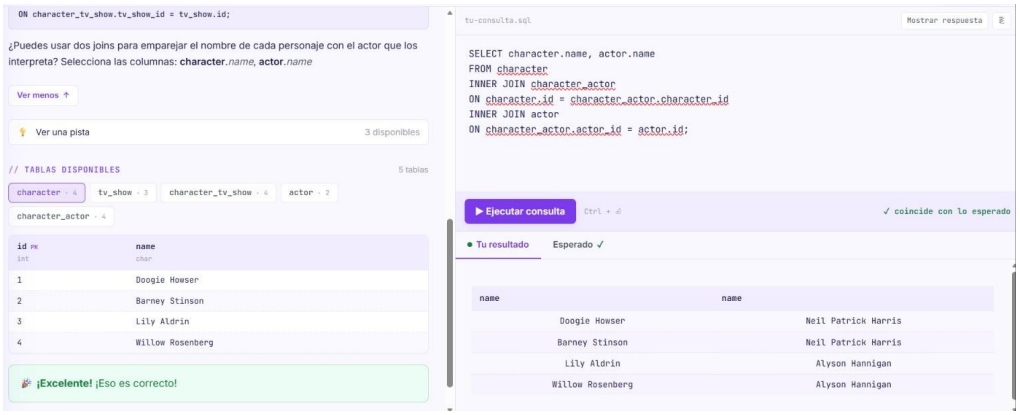
01	Ejercicio 18: Consultas anidadas
02 Objetivo	Devolver a los miembros que leyeron la mayor cantidad de libros.
03 Concepto SQL	Subconsulta: un SELECT dentro de otro SELECT.
04 Consulta	`SELECT * FROM family_members WHERE num_books_read = (SELECT MAX(num_books_read) FROM family_members);`
05 Resultado	
06 Interpretación	Devuelve la fila completa de Dave, con 200 libros. Primero se calcula el máximo y luego se buscan las filas con ese valor.
07 Aprendizaje	La consulta de adentro se ejecuta primero y su resultado se usa en la de afuera. Así resolví lo que en la lección 16 no se podía.
08 Aplicación en el mundo real	Encontrar al empleado mejor pagado o el pedido de mayor importe con todos sus datos.

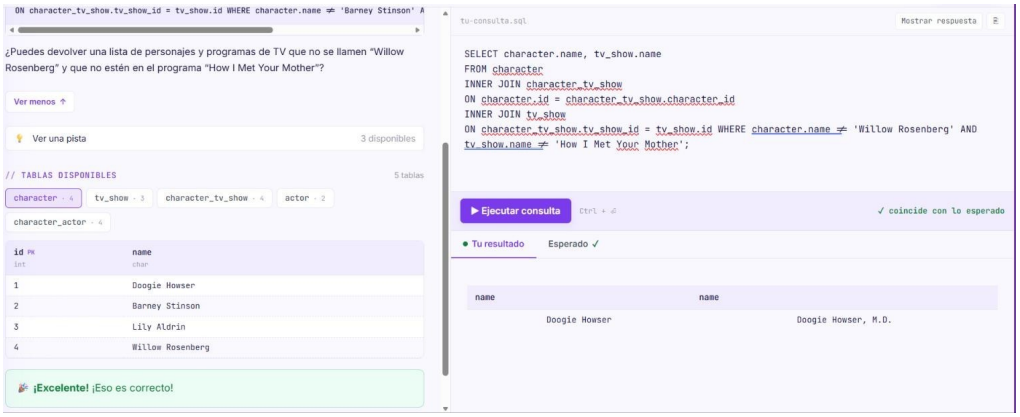
01	Ejercicio 19: NULL
02 Objetivo	Mostrar los miembros que tienen un libro favorito cargado.
03 Concepto SQL	IS NULL e IS NOT NULL para detectar valores vacíos.
04 Consulta	`SELECT * FROM family_members WHERE favorite_book IS NOT NULL;`
05 Resultado	
06 Interpretación	Devuelve a Dave y Mary. Pickles queda afuera porque su favorite_book es NULL.
07 Aprendizaje	NULL significa ausencia de dato y no se compara con =; hay que usar IS NULL o IS NOT NULL.
08 Aplicación en el mundo real	Detectar registros incompletos, por ejemplo clientes sin teléfono o sin correo electrónico.

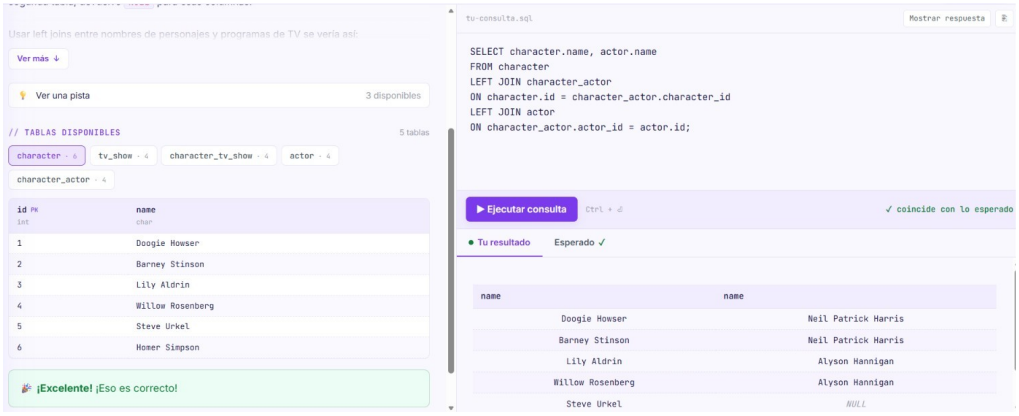
01	Ejercicio 20: Fecha
02 Objetivo	Obtener las celebridades nacidas después del 1 de septiembre de 1980.
03 Concepto SQL	Comparación de fechas con el formato AAAA-MM-DD.
04 Consulta	`SELECT * FROM celebs_born WHERE birthdate > '1980-09-01';`
05 Resultado	
06 Interpretación	Devuelve a Justin Timberlake (1981-01-31) y Taylor Swift (1989-12-13). Michael Jordan (1963) queda excluido.
07 Aprendizaje	Las fechas se escriben año-mes-día y entre comillas, y se pueden comparar con < y >.
08 Aplicación en el mundo real	Filtrar registros de un período, como los accesos de la última semana en un log de seguridad.

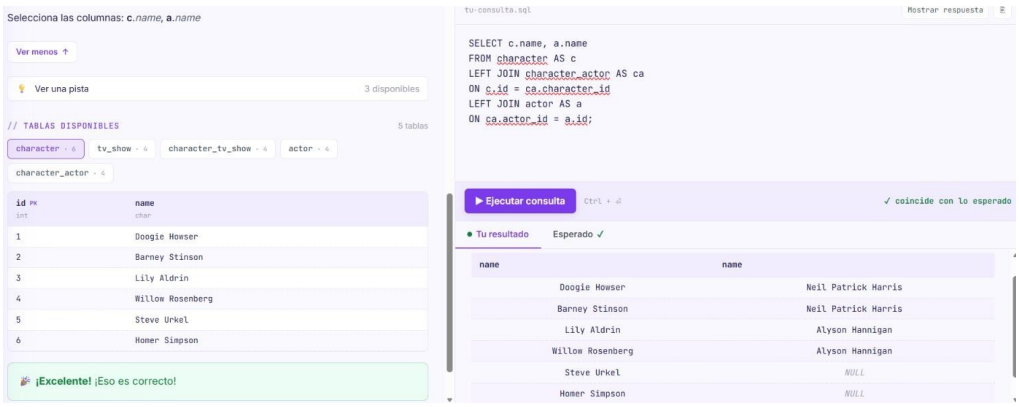
Bloque: Joins

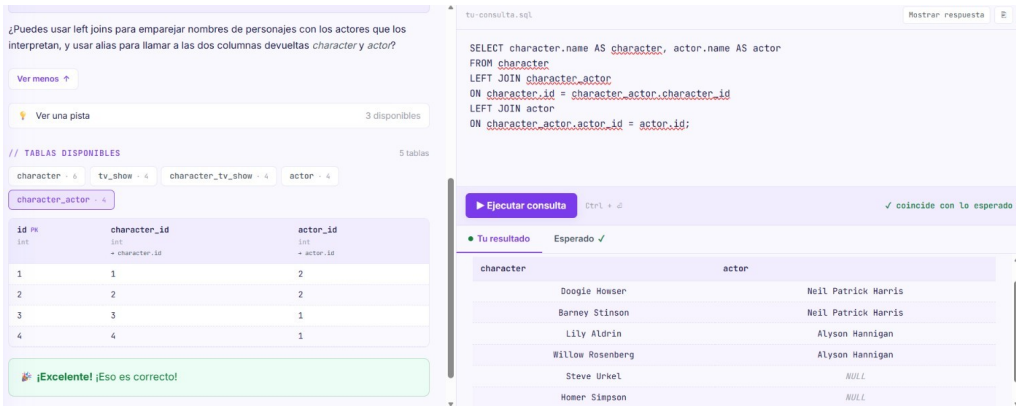
01	Ejercicio 21: Inner joins
02 Objetivo	Emparejar cada personaje con el actor que lo interpreta.
03 Concepto SQL	INNER JOIN ... ON para unir dos tablas por una columna en común.
04 Consulta	`SELECT character.name, character_actor.actor_name FROM character INNER JOIN character_actor ON character.id = character_actor.character_id;`
05 Resultado	
06 Interpretación	Muestra una fila por personaje con el nombre del actor, por ejemplo Doogie Howser con Neil Patrick Harris.
07 Aprendizaje	Los datos están repartidos en tablas relacionadas por ids; el JOIN los junta usando character.id = character_id.
08 Aplicación en el mundo real	Unir la tabla de pedidos con la de clientes para ver quién hizo cada compra.

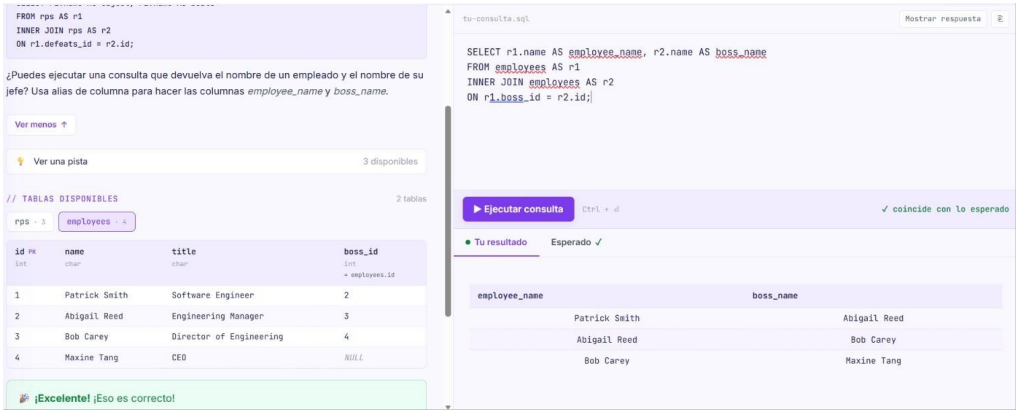
01	Ejercicio 22: Múltiples joins
02 Objetivo	Unir personajes con sus actores usando las tablas de relación.
03 Concepto SQL	Varios INNER JOIN encadenados a través de una tabla intermedia.
04 Consulta	`SELECT character.name, actor.name FROM character INNER JOIN character_actor ON character.id = character_actor.character_id INNER JOIN actor ON character_actor.actor_id = actor.id;`
05 Resultado	
06 Interpretación	El resultado es el mismo emparejamiento anterior, pero ahora el nombre del actor viene de la tabla actor, sin duplicarlo.
07 Aprendizaje	Una tabla de relación permite que un actor tenga varios personajes sin repetir su nombre en todos lados.
08 Aplicación en el mundo real	Cruzar usuarios, roles y permisos, que suelen estar en tres tablas distintas en un sistema real.

01	Ejercicio 23: Joins con WHERE
02 Objetivo	Listar personajes y programas excluyendo a Willow Rosenberg y a How I Met Your Mother.
03 Concepto SQL	JOIN combinado con WHERE y el operador !=.
04 Consulta	`SELECT character.name, tv_show.name FROM character INNER JOIN character_tv_show ON character.id = character_tv_show.character_id INNER JOIN tv_show ON character_tv_show.tv_show_id = tv_show.id WHERE character.name != 'Willow Rosenberg' AND tv_show.name != 'How I Met Your Mother';`
05 Resultado	
06 Interpretación	Queda una sola fila: Doogie Howser con Doogie Howser, M.D. El resto se descarta por el nombre o por el programa.
07 Aprendizaje	El WHERE se aplica sobre el resultado del JOIN, así que puede filtrar por columnas de cualquiera de las tablas unidas.
08 Aplicación en el mundo real	Armar un reporte de ventas de varios productos pero dejando afuera cierta región o cierto cliente.

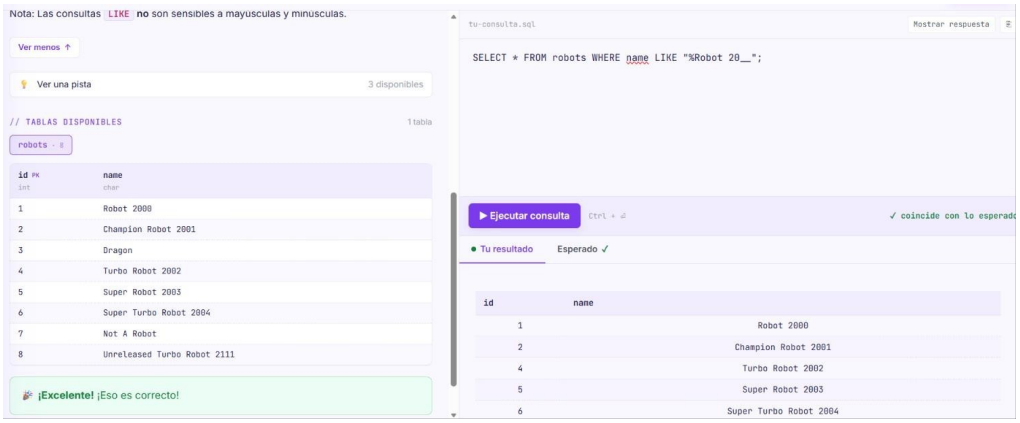
01	Ejercicio 24: Left joins
02 Objetivo	Mostrar todos los personajes aunque no tengan actor asignado.
03 Concepto SQL	LEFT JOIN: conserva todas las filas de la tabla izquierda.
04 Consulta	`SELECT character.name, actor.name FROM character LEFT JOIN character_actor ON character.id = character_actor.character_id LEFT JOIN actor ON character_actor.actor_id = actor.id;`
05 Resultado	
06 Interpretación	Aparecen todos los personajes. Steve Urkel y Homer Simpson figuran con NULL porque no tienen actor cargado.
07 Aprendizaje	A diferencia del INNER JOIN, el LEFT JOIN no pierde filas sin pareja, y rellena con NULL lo que falta.
08 Aplicación en el mundo real	Encontrar clientes que todavía no hicieron ninguna compra, o equipos sin responsable asignado.

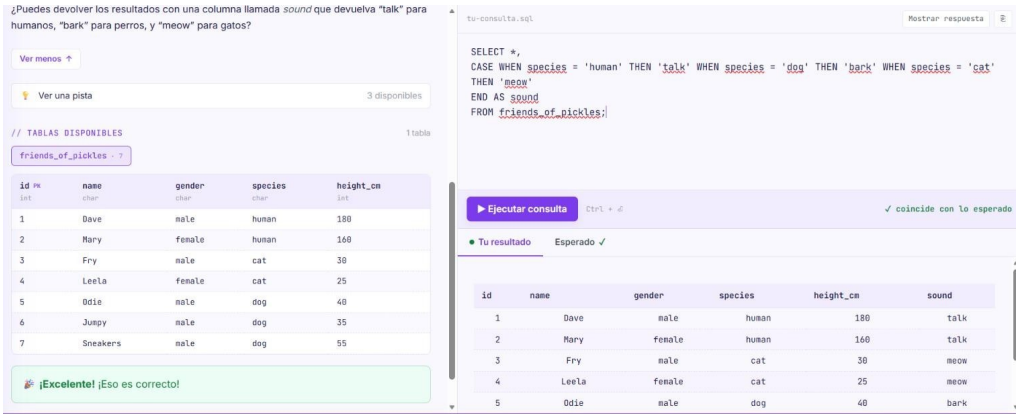
01	Ejercicio 25: Alias de tabla
02 Objetivo	Repetir el left join de personajes y actores con una consulta más corta.
03 Concepto SQL	Alias de tabla con AS (c, ca y a).
04 Consulta	`SELECT c.name, a.name FROM character AS c LEFT JOIN character_actor AS ca ON c.id = ca.character_id LEFT JOIN actor AS a ON ca.actor_id = a.id;`
05 Resultado	
06 Interpretación	Da exactamente el mismo resultado que la lección anterior, pero la consulta es más corta y se lee mejor.
07 Aprendizaje	Los alias son apodos para las tablas: no cambian el resultado, solo hacen la consulta más cómoda.
08 Aplicación en el mundo real	En bases de datos grandes con nombres de tablas largos, los alias ayudan a que las consultas sean legibles.

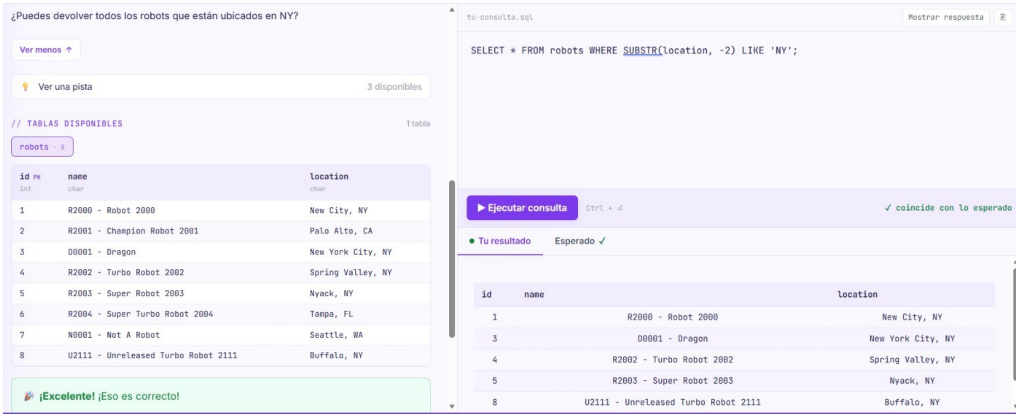
01	Ejercicio 26: Alias de columna
02 Objetivo	Renombrar las dos columnas del resultado como character y actor.
03 Concepto SQL	Alias de columna con AS.
04 Consulta	`SELECT character.name AS character, actor.name AS actor FROM character LEFT JOIN character_actor ON character.id = character_actor.character_id LEFT JOIN actor ON character_actor.actor_id = actor.id;`
05 Resultado	
06 Interpretación	Las dos columnas ahora se llaman character y actor, en lugar de ser ambas "name", lo que evita la confusión.
07 Aprendizaje	El alias cambia solo el título de la columna en el resultado, no el nombre real en la base.
08 Aplicación en el mundo real	Preparar reportes con encabezados claros para quienes no conocen la estructura interna de la base.

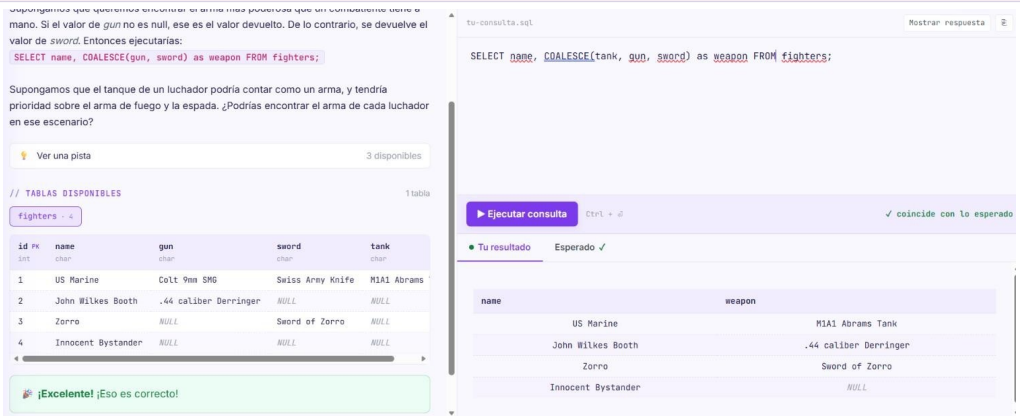
01	Ejercicio 27: Self joins
02 Objetivo	Mostrar cada empleado junto al nombre de su jefe.
03 Concepto SQL	Self join: unir una tabla consigo misma usando alias.
04 Consulta	`SELECT r1.name AS employee_name, r2.name AS boss_name FROM employees AS r1 INNER JOIN employees AS r2 ON r1.boss_id = r2.id;`
05 Resultado	
06 Interpretación	Muestra tres filas, por ejemplo Patrick Smith con Abigail Reed. Maxine Tang (CEO) no aparece porque no tiene jefe.
07 Aprendizaje	La misma tabla cumple dos roles (empleado y jefe), por eso necesito dos alias. La relación es boss_id con id.
08 Aplicación en el mundo real	Representar jerarquías: organigramas, categorías con subcategorías o comentarios con respuestas.

Bloque: Funciones de texto y lógica

01	Ejercicio 28: LIKE
02 Objetivo	Encontrar los robots cuyo nombre termina en Robot seguido de un año entre 2000 y 2099.
03 Concepto SQL	LIKE con los comodines % y _.
04 Consulta	`SELECT * FROM robots WHERE name LIKE "%Robot 20__";`
05 Resultado	
06 Interpretación	Devuelve Robot 2000, Champion Robot 2001, Turbo Robot 2002, Super Robot 2003 y Super Turbo Robot 2004. Queda afuera el de 2111.
07 Aprendizaje	% reemplaza cualquier cantidad de caracteres y _ exactamente uno. Escribí 20__ para fijar el año de 4 dígitos. Aunque la plataforma aceptó el patrón entre comillas dobles, la convención SQL para el texto es usar comillas simples: LIKE '%Robot 20__'.
08 Aplicación en el mundo real	Buscar usuarios cuyo correo termina en cierto dominio, o detectar patrones sospechosos en logs.

01	Ejercicio 29: CASE
02 Objetivo	Agregar una columna sound con talk, bark o meow según la especie.
03 Concepto SQL	Expresión CASE WHEN ... THEN ... END.
04 Consulta	`SELECT *, CASE WHEN species = 'human' THEN 'talk' WHEN species = 'dog' THEN 'bark' WHEN species = 'cat' THEN 'meow' END AS sound FROM friends_of_pickles;`
05 Resultado	
06 Interpretación	Cada fila recibe un sonido según su especie: talk para humanos, bark para perros y meow para gatos.
07 Aprendizaje	CASE funciona como un if dentro de la consulta. Sin ELSE, cualquier especie no contemplada daría NULL. Con ELSE se define un valor por defecto para esos casos, por ejemplo ELSE 'unknown'.
08 Aplicación en el mundo real	Clasificar ventas en alta, media o baja, o etiquetar niveles de riesgo según un puntaje.

01	Ejercicio 30: SUBSTR
02 Objetivo	Obtener los robots ubicados en NY.
03 Concepto SQL	SUBSTR con índice negativo para tomar los últimos caracteres.
04 Consulta	`SELECT * FROM robots WHERE SUBSTR(location, -2) LIKE 'NY';`
05 Resultado	
06 Interpretación	Devuelve los robots cuya ubicación termina en NY, como New York City, Spring Valley, Nyack y Buffalo.
07 Aprendizaje	Con índice -2 se toman los dos últimos caracteres del texto. Para comparar un valor exacto, también serviría = 'NY'.
08 Aplicación en el mundo real	Extraer partes de un dato, como el código de área de un teléfono o el dominio de un correo.

01	Ejercicio 31: COALESCE
02 Objetivo	Obtener el arma de cada luchador dando prioridad a tank, luego gun y luego sword.
03 Concepto SQL	COALESCE: devuelve el primer valor que no es NULL.
04 Consulta	`SELECT name, COALESCE(tank, gun, sword) AS weapon FROM fighters;`
05 Resultado	
06 Interpretación	US Marine usa el tanque, John Wilkes Booth la pistola, Zorro la espada e Innocent Bystander queda en NULL porque no tiene nada.
07 Aprendizaje	El orden de los argumentos define la prioridad. Si todos son NULL, el resultado también lo es. Un NULL no equivale a una cadena vacía: NULL es ausencia de dato, mientras que '' es un texto vacío que COALESCE trata como un valor válido.
08 Aplicación en el mundo real	Mostrar un teléfono de contacto alternativo cuando falta el principal, o reemplazar valores vacíos por uno por defecto.

Reflexión y conclusión general

Temas que me resultaron más fáciles

Los fundamentos (SELECT, WHERE, ORDER BY y LIMIT) fueron los más directos: las consultas se parecen bastante a frases en inglés y sus resultados se comprenden fácilmente. Por su parte, las funciones de agregación (COUNT, SUM, AVG, MAX y MIN) me resultaron sencillas de analizar y comprender, ya que devuelven un único valor que resume los datos de una tabla, lo que facilita la interpretación de los resultados.

Temas que me resultaron más desafiantes

Los joins fueron los más exigentes, sobre todo los múltiples joins con tablas de relación y el self join, porque hay que pensar en cómo se conectan las tablas y no solo en qué columnas pedir. También me costó pensar el LEFT JOIN, ya que aparecen valores NULL que hay que interpretar. Lo resolví dibujando las tablas y siguiendo qué id se une con cuál.

Cómo se aplican estas consultas en un contexto real

Estas consultas son la base del trabajo con datos en cualquier organización: filtrar clientes, resumir ventas por categoría, unir pedidos con usuarios o detectar registros incompletos. En ciberseguridad se usan para analizar logs, contar intentos de acceso fallidos, ordenar alertas por gravedad y cruzar usuarios con permisos. Además, entender SQL ayuda a comprender cómo funciona una inyección SQL y a distinguirla de otras vulnerabilidades web, como el Cross-Site Scripting (XSS) que analizo en la segunda parte del desafío con OWASP ZAP.

Conclusión

Completar los ejercicios me permitió avanzar desde consultas sencillas sobre una sola tabla hasta operaciones que permiten combinar información de varias tablas, resumirla y transformarla. A lo largo de la práctica, fui comprendiendo cómo utilizar las distintas cláusulas y funciones de SQL para obtener la información que necesitaba y la importancia de escribir consultas claras y precisas.

También aprendí que algunos aspectos, como el tratamiento de los valores NULL y las relaciones entre tablas, requieren analizar cuidadosamente cada situación para interpretar correctamente los resultados. Resolver los ejercicios me ayudó a familiarizarme con la sintaxis de SQL, reconocer errores y comprender mejor la lógica detrás de cada consulta.

Además, pude reconocer que SQL tiene aplicaciones que van más allá de las consultas practicadas. Es una herramienta importante para gestionar y analizar datos en distintos tipos de organizaciones y también puede contribuir al análisis de información relevante para la seguridad informática. Comprender cómo se construyen las consultas permite, a su vez, entender mejor los riesgos como la inyección SQL y la importancia de utilizar mecanismos que protejan los datos.

En conclusión, esta práctica me permitió fortalecer mis conocimientos y adquirir una base para seguir profundizando en SQL, abordar consultas más complejas y aplicar lo aprendido en futuros proyectos relacionados con el análisis, la gestión y la seguridad de los datos.

Parte 2: Análisis de seguridad web con OWASP ZAP

Primera evaluación de seguridad web de Gin & Juice Shop

01	Datos de la evaluación
02 Fecha de las pruebas	22/10/2026
03 Herramienta	OWASP ZAP 2.17.0 (Zed Attack Proxy)
04 Entorno	Máquina virtual con Kali Linux · navegador Firefox
05 Objetivo	https://ginandjuice.shop/ (sitio de práctica)

Alcance y método

Este trabajo documenta una primera evaluación de seguridad web hecha con OWASP ZAP sobre **Gin & Juice Shop**, una tienda creada por PortSwigger para practicar con escáneres de vulnerabilidades. El propio sitio lo indica en un aviso que se ve en la captura del carrito (Figura 2.4). Las pruebas se hicieron solo sobre este sitio y con fines educativos.

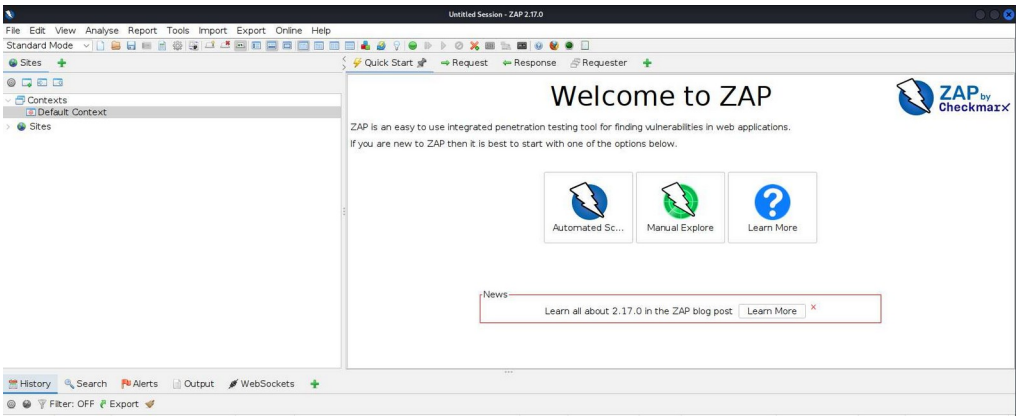
El procedimiento fue el siguiente:

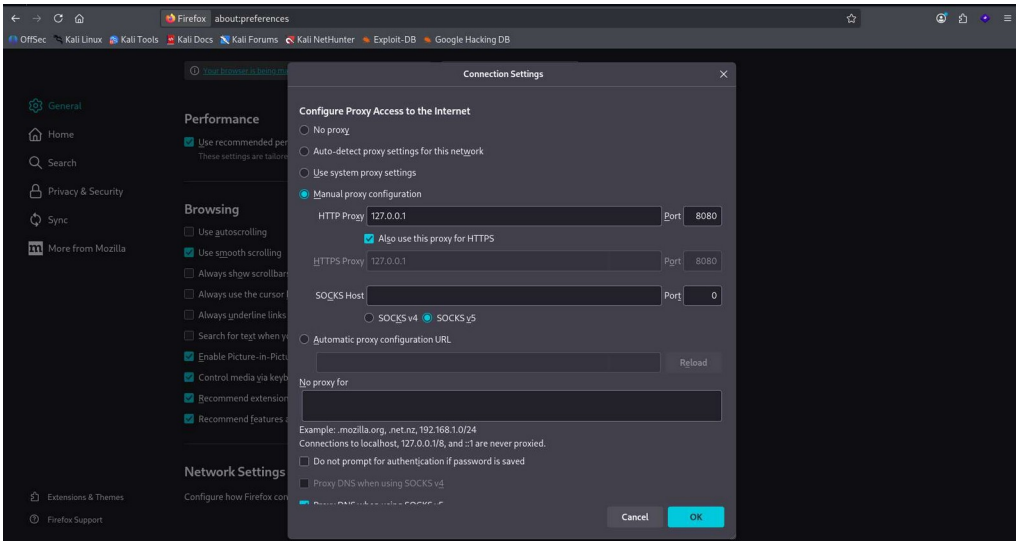
- **Preparación:** ZAP funcionando como proxy local (127.0.0.1:8080) y Firefox configurado para enviar su tráfico por ese proxy.
- **Exploración manual:** navegué por el sitio (inicio, productos, carrito e inicio de sesión) mientras ZAP registraba el tráfico y lo analizaba de forma pasiva.
- **Spider:** ejecuté el Spider de ZAP para descubrir más rutas y recursos del sitio.
- **Active Scan:** ejecuté el escaneo activo, que envía entradas de prueba a los parámetros para detectar vulnerabilidades.
- **Análisis de alertas:** revisé la pestaña Alerts, elegí tres alertas de distinto nivel y documenté un hallazgo principal con una ficha técnica.

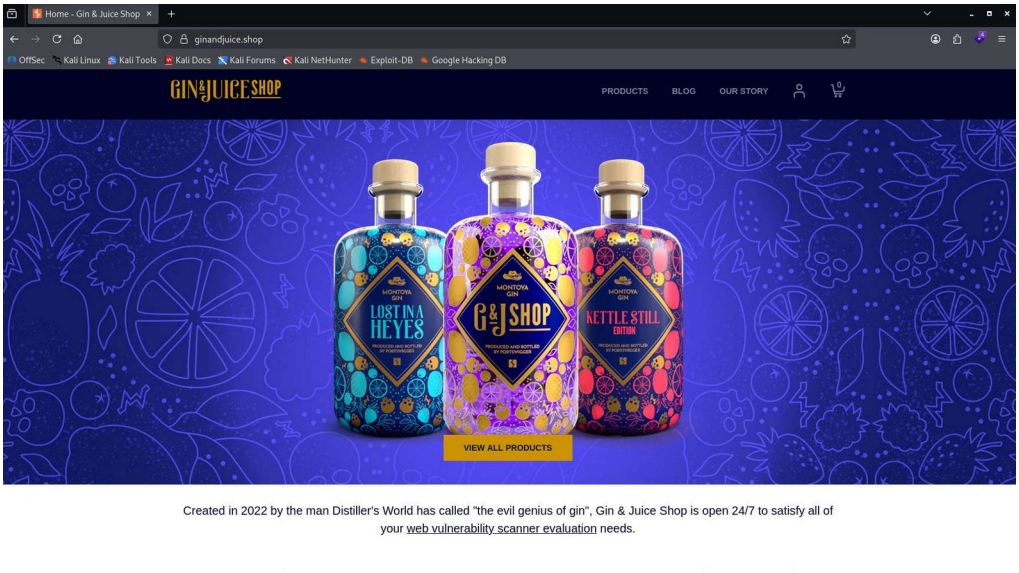
Limitación: el Active Scan se detuvo antes de completarse (ver las conclusiones), por lo que la lista de alertas corresponde a un análisis parcial del sitio. Además, los hallazgos son detecciones de ZAP y todavía no se validaron manualmente.

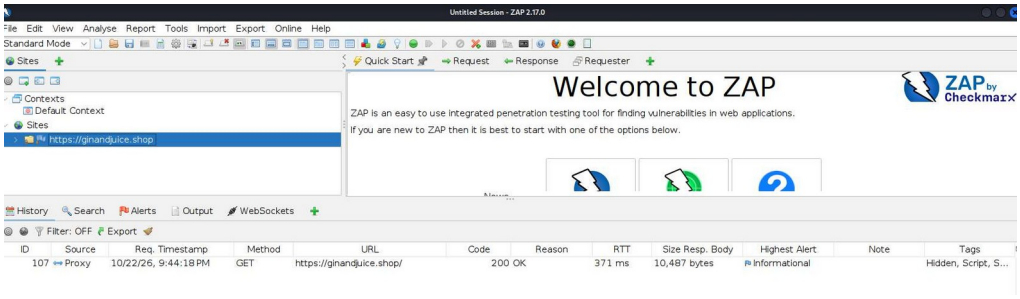
Bloque 1: Preparación del entorno

Se documentan 4 elementos

01	1.1 ZAP abierto
02 Captura	
03 Descripción	**Figura 1.1** · Interfaz principal de OWASP ZAP 2.17.0, con los paneles Sites e History todavía vacíos.

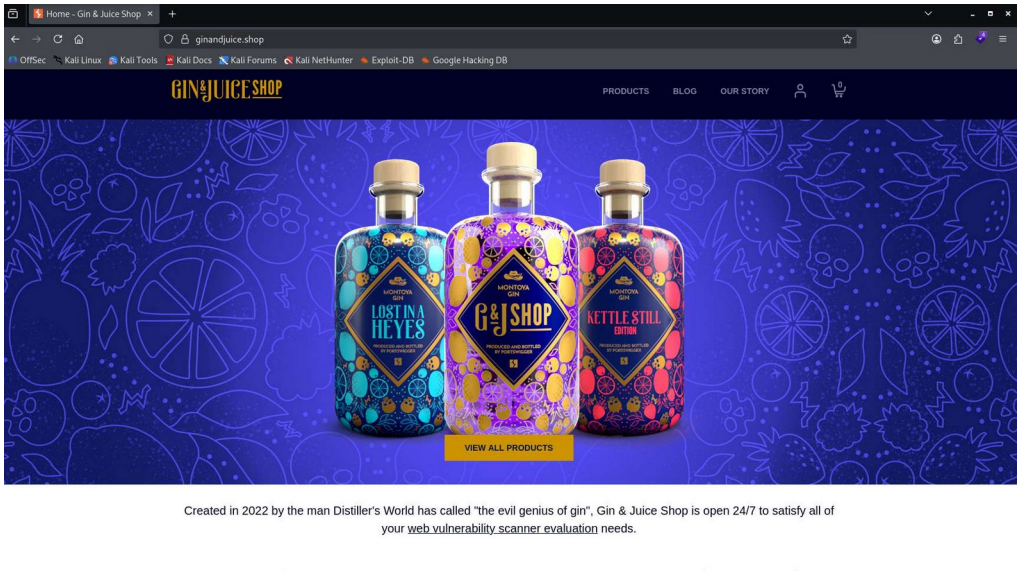
01	1.2 Navegador configurado
02 Captura	
03 Descripción	**Figura 1.2** · Firefox con proxy manual 127.0.0.1, puerto 8080, y la opción «Also use this proxy for HTTPS» activada.
04 Detalle	Para poder analizar el tráfico HTTPS, Firefox mostró primero un error de conexión porque no confiaba en el certificado raíz de ZAP. Lo resolví exportando el certificado desde ZAP (Tools → Options → Network → Server Certificates) e importándolo en Firefox como autoridad certificadora.

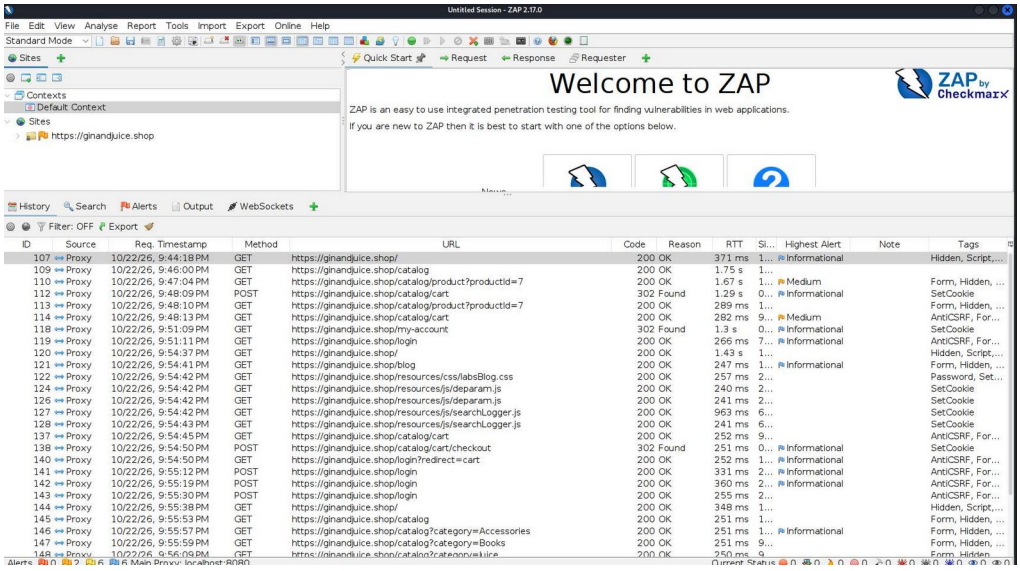
01	1.3 Acceso a https://ginandjuice.shop/
02 Captura	
03 Descripción	**Figura 1.3** · Acceso al sitio desde Firefox, con la conexión segura (candado) y ya pasando por el proxy de ZAP.

01	1.4 Verificación de que el tráfico aparece en ZAP
02 Captura	
03 Descripción	**Figura 1.4** · La pestaña History registra la petición **GET https://ginandjuice.shop/** (código 200): el tráfico del navegador pasa por ZAP.

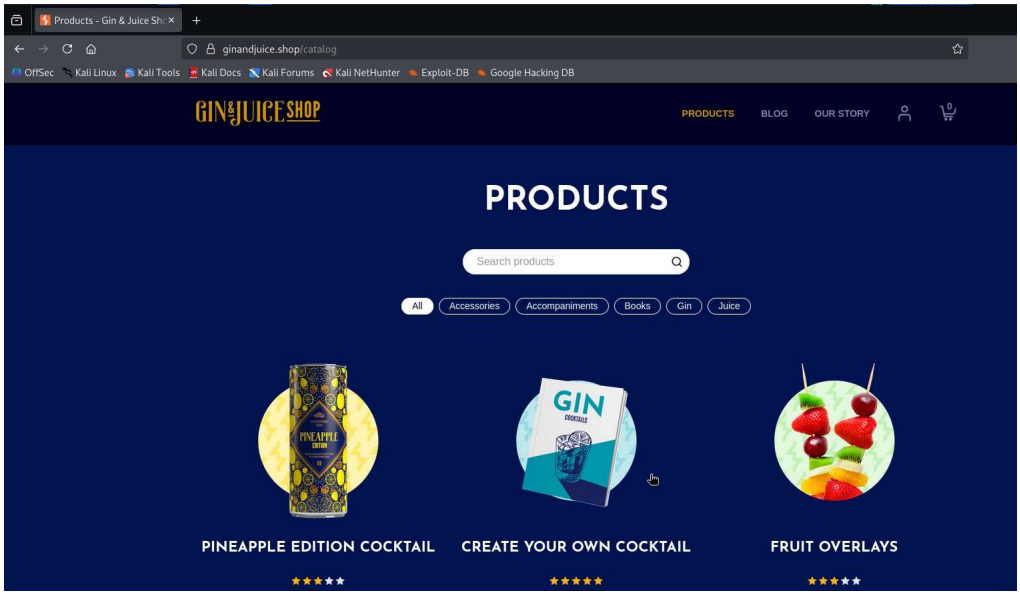
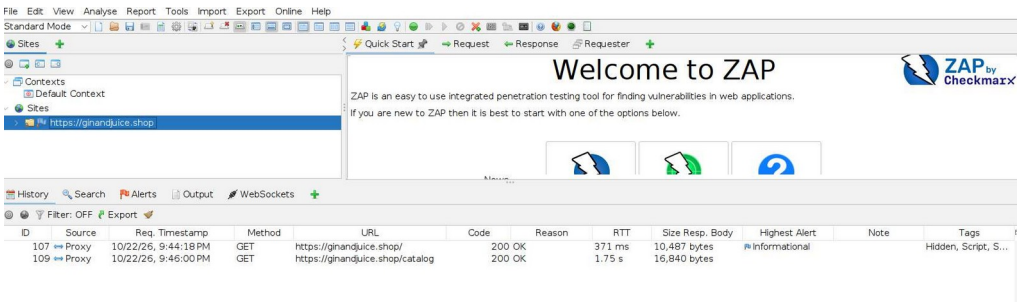
Bloque 2: Exploración de la aplicación

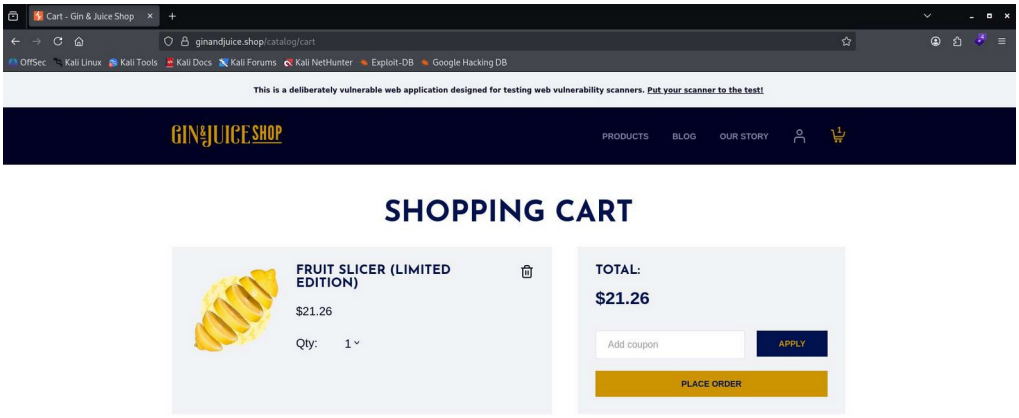
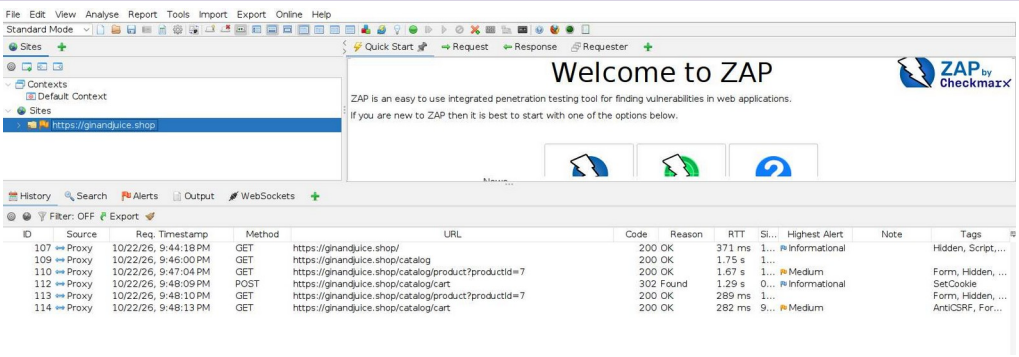
Se documentan 5 elementos: navegación, tráfico y 3 secciones más

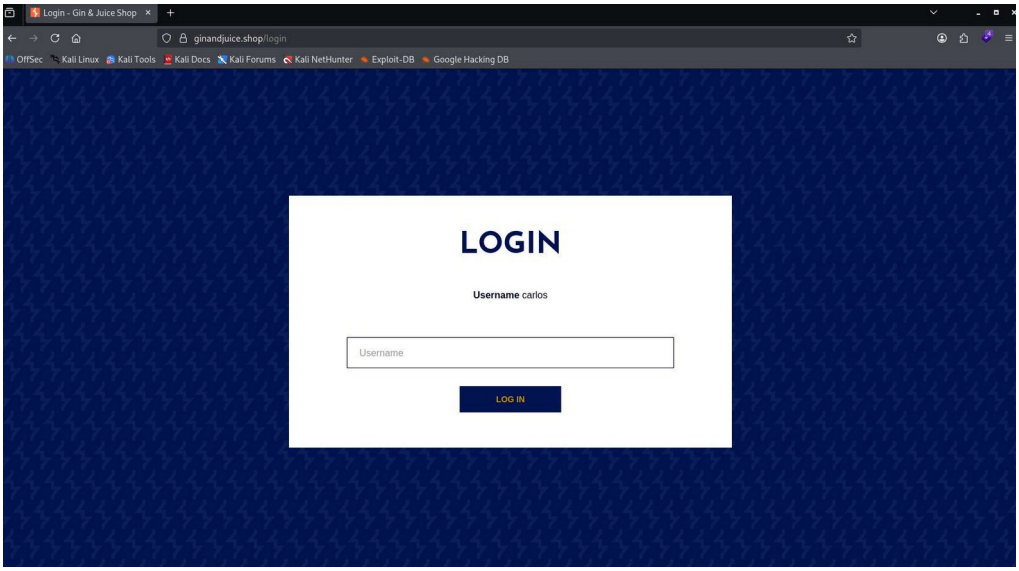
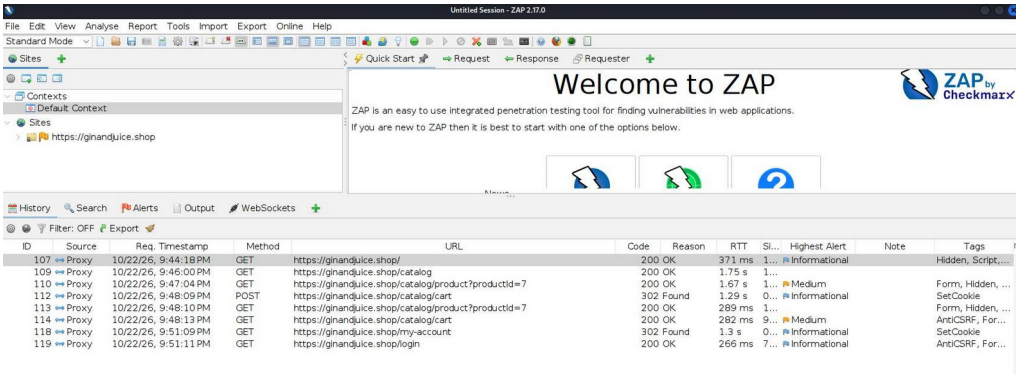
01	2.1 Navegación en el sitio
02 Captura	
03 Descripción	**Figura 2.1** · Página principal de Gin & Juice Shop.

01	2.2 Tráfico HTTP en ZAP
02 Captura	
03 Descripción	**Figura 2.2** · History con las peticiones generadas al navegar: GET y POST a /catalog, /catalog/product, /catalog/cart, /my-account y /login, además de recursos estáticos (JS y CSS). Cada fila muestra el código de respuesta, el tiempo y la alerta más alta asociada.

Otras secciones exploradas

01	2.3 Sección de productos
02 Captura	
03 Descripción	**Figura 2.3** · Catálogo de productos (/catalog) con buscador y filtros por categoría.
04 Tráfico en ZAP	
05 Descripción del tráfico	**Figura 2.3b** · Tráfico generado: **GET /catalog** (200 OK).

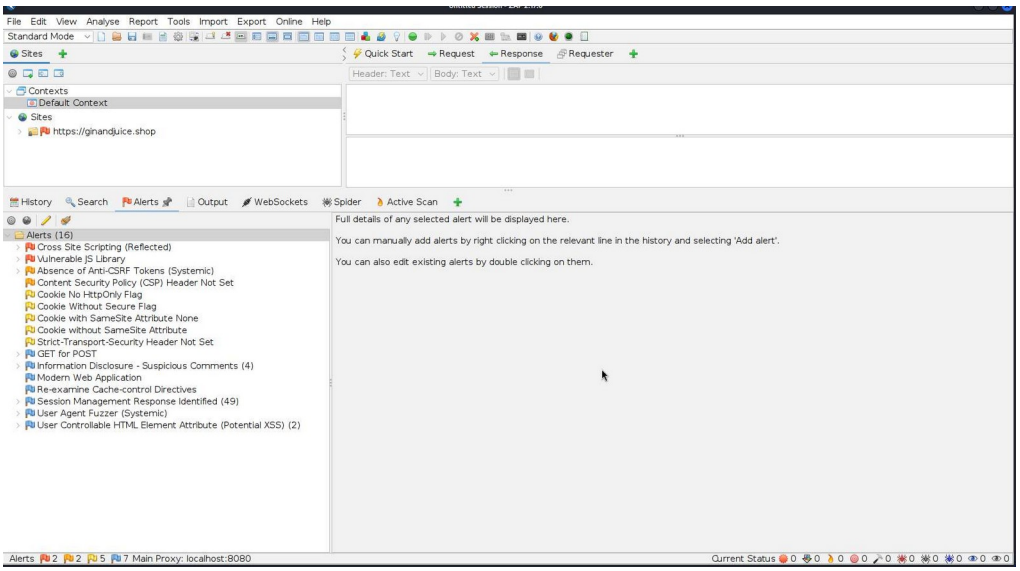
01	2.4 Carrito de compras
02 Captura	
03 Descripción	**Figura 2.4** · Carrito con un producto (Fruit Slicer, $21.26). Arriba se ve el aviso de que el sitio es deliberadamente vulnerable.
04 Tráfico en ZAP	
05 Descripción del tráfico	**Figura 2.4b** · Tráfico generado: **POST /catalog/cart** (302 Found) al agregar el producto y **GET /catalog/cart** (200 OK), esta última con una alerta de nivel Medium (etiqueta AntiCSRF).

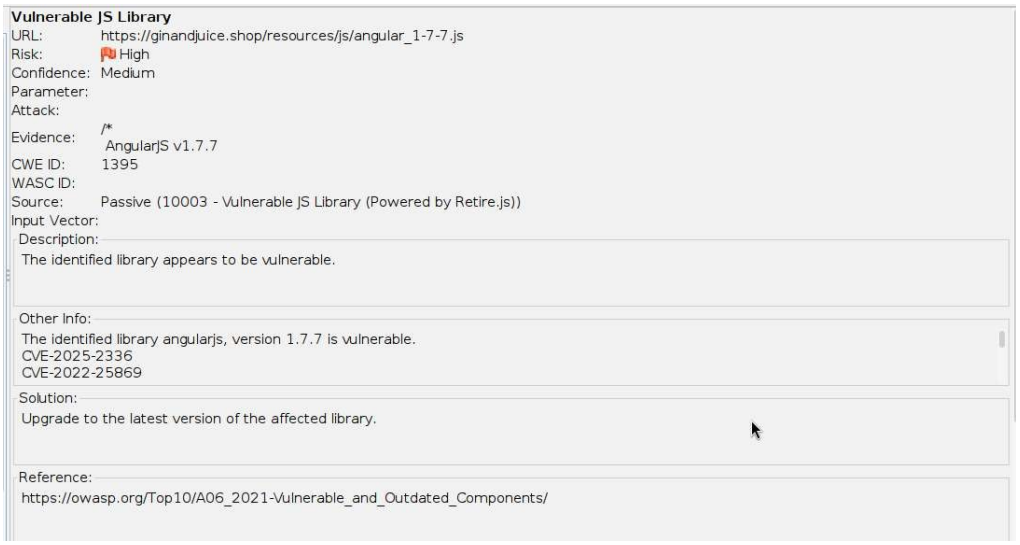
01	2.5 Inicio de sesión
02 Captura	
03 Descripción	**Figura 2.5** · Página de inicio de sesión (/login).
04 Tráfico en ZAP	
05 Descripción del tráfico	**Figura 2.5b** · Tráfico generado: **GET /my-account** (302 Found) y **GET /login** (200 OK), con etiqueta AntiCSRF.

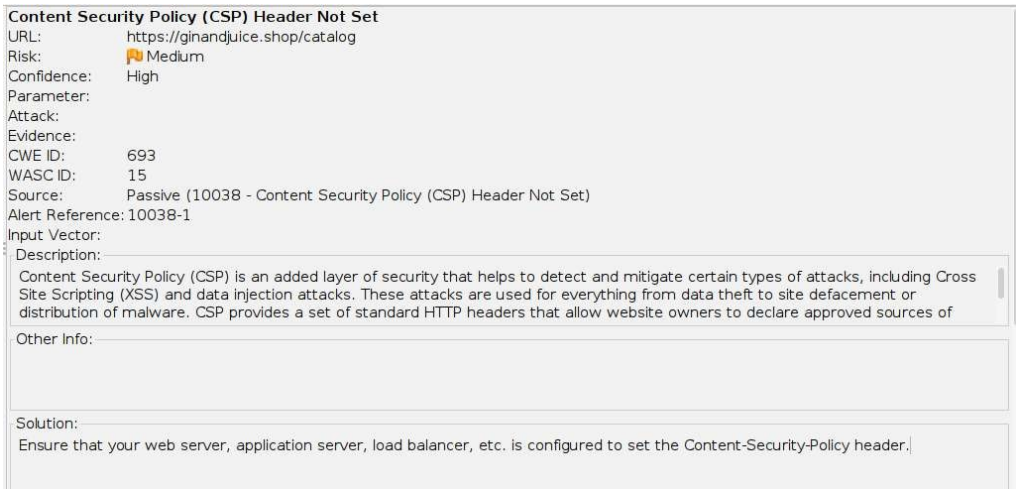
Bloque 3: Análisis de alertas

Se documentan 3 alertas de tipos y niveles diferentes

Después de navegar, ejecutar el Spider y el Active Scan, la pestaña Alerts de ZAP listó **16 tipos de alerta**: 2 de riesgo alto (bandera roja), 2 medio (naranja), 5 bajo (amarilla) y 7 informativas (azul). ZAP agrupa las alertas por tipo y cada grupo se despliega en las URL afectadas.

01	Lista de alertas detectadas por ZAP
02 Captura	
03 Descripción	**Figura 3.1** · Lista de alertas de ZAP (Alerts 16). La barra inferior resume 2 altas, 2 medias, 5 bajas y 7 informativas.

01	Alerta 1 · Vulnerable JS Library
02 Nivel de riesgo	**Alto**
03 Captura	
04 Descripción	**Figura 3.2** · Detalle de la alerta Vulnerable JS Library.
05 Análisis	ZAP detectó, de forma pasiva, que el sitio carga **AngularJS v1.7.7** desde */resources/js/angular_1-7-7.js* y que esa versión tiene vulnerabilidades conocidas (CWE-1395, categoría OWASP A06:2021, componentes vulnerables y desactualizados). La confianza es media y no hay un parámetro afectado, porque el problema es el componente en sí. La solución indicada es actualizar la librería. Como AngularJS 1.x ya no recibe soporte oficial, conviene además evaluar su reemplazo. Detectar una versión con vulnerabilidades conocidas justifica priorizar su actualización o reemplazo y evaluar los CVE que apliquen según la versión, la configuración y las funcionalidades utilizadas, pero por sí solo no demuestra que todas puedan explotarse en este sitio.
06 CVE asociados (13)	CVE-2025-2336, CVE-2022-25869, CVE-2022-25844, CVE-2024-21490, CVE-2025-0716, CVE-2024-8373, CVE-2024-8372, CVE-2023-26117, CVE-2019-10768, CVE-2023-26118, CVE-2023-26116, CVE-2020-7676, CVE-2025-4690
07 Etiquetas (Tags)	CWE-1395 · OWASP_2021_A06 (Vulnerable and Outdated Components) · OWASP_2017_A09 (Using Components with Known Vulnerabilities)
08 Referencias	https://nvd.nist.gov/vuln/detail/CVE-2025-2336 https://nvd.nist.gov/vuln/detail/CVE-2022-25869 https://nvd.nist.gov/vuln/detail/CVE-2022-25844 https://owasp.org/Top10/A06_2021-Vulnerable_and_Outdated_Components/ https://docs.angularjs.org/misc/version-support-status

01	Alerta 2 · Content Security Policy (CSP) Header Not Set
02 Nivel de riesgo	Medio
03 Captura	
04 Descripción	**Figura 3.3** · Detalle de la alerta CSP Header Not Set.
05 Análisis	La respuesta de *https://ginandjuice.shop/catalog* no incluye la cabecera **Content-Security-Policy** (confianza alta, CWE-693, regla pasiva 10038). CSP es una capa de seguridad adicional que permite declarar desde qué fuentes puede cargarse contenido y ayuda a mitigar ataques como Cross Site Scripting e inyección de datos. La solución es configurar el servidor web, el servidor de aplicaciones o el balanceador para que envíe la cabecera. Está relacionada con el hallazgo principal: sin CSP, un XSS encuentra menos barreras, aunque una CSP no reemplaza la corrección del problema original.

01	Alerta 3 · Information Disclosure - Suspicious Comments
02 Nivel de riesgo	Informativo
03 Captura	
04 Descripción	**Figura 3.4** · Detalle de la alerta Information Disclosure - Suspicious Comments.
05 Análisis	En */resources/js/react.development.js* ZAP encontró un comentario que coincide con el patrón de palabras sospechosas (evidencia: **bugs**; el comentario dice «// No known bugs, but needs performance testing»). La confianza es baja (CWE-615, regla pasiva 10027) y el riesgo es solo informativo, porque el texto parece inofensivo. Aun así, ZAP recomienda quitar los comentarios que puedan dar pistas a un atacante, y el nombre del archivo sugiere que el sitio sirve una versión de desarrollo de React, algo que conviene revisar.

Bloque 4: Hallazgo principal: Cross Site Scripting (Reflected)

Ficha técnica de 8 elementos

01	Cross Site Scripting (Reflected)
02 Captura	
03 Descripción	**Figura 4.1** · Detalle de la alerta Cross Site Scripting (Reflected) en ZAP: URL, parámetro, ataque y evidencia.

Ficha técnica	Cross Site Scripting (Reflected)
01 Nombre de la alerta	Cross Site Scripting (Reflected), también llamado XSS reflejado.
02 Nivel de riesgo	**Alto** (confianza media).
03 URL o componente	https://ginandjuice.shop/login (formulario de inicio de sesión, petición POST).
04 Parámetro afectado	`username` (vector de entrada: Form Query).
05 Evidencia	Payload enviado por ZAP y devuelto en la respuesta: `';alert(1);'` (ver Figura 4.1). **Estado de validación:** alerta detectada por ZAP; confirmación manual pendiente.
06 Descripción técnica	Durante el Active Scan, ZAP envió el payload en el parámetro *username* y registró su reflejo en la respuesta sin codificarlo. Es un indicio de que la entrada podría no estar codificándose correctamente, pero que una cadena se refleje no demuestra por sí solo que el navegador la ejecute como código: para confirmarlo hay que revisar el contexto HTML o JavaScript en el que se refleja y hacer una prueba manual controlada. Es un XSS de tipo reflejado: el código viaja en la petición y vuelve en la respuesta, de modo que la víctima tendría que abrir un enlace o enviar un formulario preparado. Fue detectado por la regla activa 40012 (confianza media), es decir, apareció gracias al Active Scan.
07 Impacto potencial	Si el hallazgo se confirmara, un atacante podría ejecutar JavaScript en el navegador de la víctima con los permisos del sitio: robar cookies de sesión y secuestrar la cuenta, redirigir a otro sitio o mostrar contenido fraudulento. En este mismo escaneo ZAP detectó también «Cookie No HttpOnly Flag», lo que podría dejar algunas cookies al alcance de un script.
08 Recomendación	Codificar la salida según el contexto (HTML, atributo o JavaScript) y usar frameworks que escapen por defecto; definir la codificación de caracteres (por ejemplo UTF-8) y marcar las cookies como HttpOnly. Como controles adicionales, validar la entrada en el servidor con una lista de valores permitidos y definir una política CSP: ayudan, pero no reemplazan la codificación de salida ni la corrección del problema original.

Complemento	Clasificación y referencias de la alerta
01 Clasificación	CWE-79 · WASC 8 · OWASP Top 10 2021: A03 Injection · OWASP Top 10 2017: A07 Cross-Site Scripting
02 Guía de pruebas	WSTG-v42-INPV-01 (Testing for Reflected Cross Site Scripting)
03 Cumplimiento	Etiquetas PCI_DSS y HIPAA de ZAP
04 Referencias	https://cwe.mitre.org/data/definitions/79.html https://owasp.org/Top10/A03_2021-Injection/ https://owasp.org/www-project-top-ten/2017/A7_2017-Cross-Site_Scripting_(XSS).html https://owasp.org/www-project-web-security-testing-guide/v42/4-Web_Application_Security_Testing/07-Input_Validation_Testing/01-Testing_for_Reflected_Cross_Site_Scripting

Conclusiones

¿Qué aprendí?

Aprendí cómo ZAP se ubica entre el navegador y el sitio para registrar y analizar todo el tráfico. Con solo navegar, el análisis pasivo ya mostró alertas de nivel medio y bajo (tokens anti-CSRF, cabecera CSP, cookies). Al sumar el Spider y el Active Scan, ZAP recorrió más rutas y recursos y aparecieron el resto de los hallazgos, entre ellos la librería AngularJS vulnerable y el XSS reflejado en el parámetro *username* de /login. También aprendí a leer cada alerta (riesgo, confianza, URL, parámetro y evidencia) y a ver que los hallazgos se relacionan: por ejemplo, la falta de CSP y las cookies sin HttpOnly agravan el impacto de un XSS.

¿Qué fue lo más retador?

- **HTTPS y el certificado:** Firefox bloqueaba la conexión porque no confiaba en el certificado raíz de ZAP. Lo resolví exportándolo desde ZAP e importándolo en Firefox.
- **Recursos de la máquina virtual:** durante el Active Scan la VM se trabó y tuve que repetir todo el proceso. Al revisar el log de arranque vi que ZAP trabajaba con unos 480 MB de memoria en una VM de aproximadamente 2 GB, lo que explica el problema. Por eso detuve el escaneo antes de que terminara y trabajé con las alertas obtenidas hasta ese momento.
- **Interpretar los resultados:** entender qué significan el riesgo y la confianza, y elegir qué alertas documentar entre las 16 que aparecieron.

¿Qué revisaría a continuación?

- Dejar que el Active Scan termine, asignando más memoria a la máquina virtual y reduciendo la cantidad de hilos por host, para completar el análisis y obtener posibles alertas adicionales.
- Probar también el AJAX Spider, ya que ZAP identificó el sitio como una aplicación web moderna, y comprobar si permite descubrir rutas o contenidos que el Spider tradicional no encontró.
- Confirmar el XSS de forma manual en este entorno controlado y revisar las demás alertas de nivel medio y bajo: tokens anti-CSRF, cookies sin Secure, HttpOnly o SameSite, y la cabecera HSTS.
- En un caso real, priorizar la corrección según el riesgo: actualizar o reemplazar AngularJS si corresponde, codificar correctamente los datos de salida y agregar una política CSP adecuada.

Resumen de hallazgos documentados

Risk Level	Name	URL	Parameter	Evidence	Estado
High	Cross Site Scripting (Reflected)	/login	`username`	`';alert(1);'`	Detectado por ZAP; validación manual pendiente
High	Vulnerable JS Library	/resources/js/angular_1-7-7.js	-	AngularJS v1.7.7	Detectado por ZAP; CVE por evaluar
Medium	Content Security Policy (CSP) Header Not Set	/catalog	-	-	Detectado por ZAP
Informational	Information Disclosure - Suspicious Comments	/resources/js/react.development.js	-	bugs	Detectado por ZAP

Total detectado por ZAP: 16 tipos de alerta (2 altas, 2 medias, 5 bajas y 7 informativas). Son tipos de alerta registrados por la herramienta, no necesariamente 16 vulnerabilidades únicas confirmadas. Se documentaron 4 de ellas: las tres del bloque 3 y el hallazgo principal del bloque 4. «Detectado por ZAP» indica que la herramienta lo registró; las alertas todavía no se validaron manualmente.